

Healthcare Insurance Fraud Detection System

MASTER PROJECT CONTEXT & BUILD INSTRUCTIONS FOR A NEW CHAT

Purpose: Upload this file at the start of a new ChatGPT conversation so the new chat can understand the existing project, current implementation, completed work, pending features, exact database structure, working backend configuration, and the required step-by-step development process.

IMPORTANT: This is an existing project. The new chat must continue from the current state and must NOT restart the project from scratch or overwrite existing working code without first inspecting it.

1. MASTER INSTRUCTION FOR THE NEW CHAT

You are continuing an existing Healthcare Insurance Fraud Detection System project.

Your job is to BUILD the project feature-by-feature, step-by-step, using the existing codebase.

RULES:

1. Do not restart the project from scratch.
2. Do not assume unfinished features are complete.
3. Before changing an existing file, ask the user to provide its current code if it has not already been provided.
4. Do not overwrite working code blindly.
5. Work one feature at a time.
6. Give exact folder paths, filenames, terminal commands, SQL commands, and complete code when needed.
7. After each feature, provide exact testing instructions.
8. Wait for the user to confirm that the current step works before moving to the next major step.
9. If an error occurs, debug the error first and do not continue to unrelated features.
10. Keep the project architecture consistent.
11. Use environment variables for secrets and never ask the user to expose their real passwords.
12. Clearly mark each item as COMPLETED, IN PROGRESS, or PENDING.
13. The current backend runs on port 5001.
14. The MySQL database name is insurance_fraud_detection.
15. The next major feature to implement is Authentication: Register + bcrypt password hashing + Login + JWT + protected Profiles.
16. Before implementing authentication, inspect the existing:
 - controllers/authController.js
 - routes/authRoutes.js
 - middleware/authMiddleware.js
17. Preserve working code and complete missing functionality instead of replacing everything unnecessarily.

Recommended workflow:

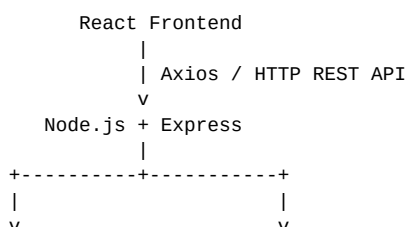
Inspect -> Explain -> Implement -> Run -> Test -> Confirm -> Continue.

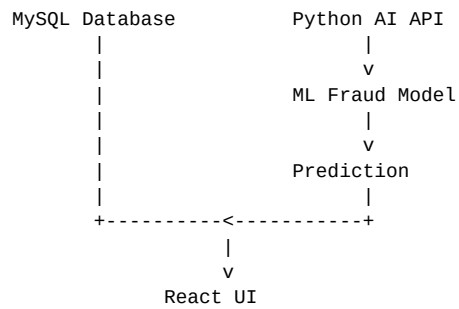
2. PROJECT OBJECTIVE

The project is an end-to-end Healthcare Insurance Fraud Detection System. The intended application allows users to register and log in, submit insurance claims, upload supporting documents, analyze claim-related information with an AI/ML fraud detection model, view fraud/genuine predictions and risk information, and allow authorized administrators to review and update claims.

The planned architecture combines a React frontend, Node.js/Express backend, MySQL database, and Python AI/ML prediction service. Blockchain is an optional later enhancement intended for integrity verification rather than storing sensitive healthcare documents directly on-chain.

3. TARGET ARCHITECTURE





Later:
Claim/Prediction Data -> Hash -> Blockchain Integrity Record

4. TECHNOLOGY STACK

Layer	Technology
Frontend	React 19
Build Tool	Vite
Routing	React Router DOM
HTTP/API	Axios
UI	Bootstrap 5 or Material UI
Icons	React Icons
Charts	Chart.js
State/Auth	Context API
Authentication	JWT
Backend	Node.js + Express.js
Database	MySQL
MySQL Driver	mysql2
Password Hashing	bcrypt
File Upload	multer
Configuration	dotenv
CORS	cors
Development	nodemon
AI/ML Service	Python
Candidate ML Models	Logistic Regression, Decision Tree, Random Forest, XGBoost
Blockchain	Planned later; integrity/hash verification

5. FRONTEND PLANNED FEATURES

The React UI was already created/planned with separate user and administrator workflows and a dedicated AI fraud detection area.

Main pages:

- Login
- Register
- Forgot Password
- Dashboard
- Submit Claim
- Claim History
- Claim Details
- Prediction Result
- AI Fraud Detection
- Profile
- Settings
- Admin Dashboard
- Not Found / Error

Reusable components planned:

- Navbar
- Sidebar
- Footer
- ProtectedRoute
- LoadingSpinner
- Card
- Table
- Modal

Application state:

- AuthContext

API communication:

- services/api.js

Dashboard planned information includes total claims, approved claims, rejected claims, fraud detected, pending claims, Fraud vs Genuine chart, Claims Per Month chart, and Recent Claims table. Submit Claim includes customer information, policy details, claim amount/type, hospital details, vehicle information where applicable, police report, incident information, description, and uploads. Prediction Result includes Fraud/Genuine prediction, confidence/probability, risk level, reasons, recommended action, and probability meter. Admin Dashboard includes claim search/filtering, user viewing, prediction viewing, approve/reject actions, deletion where appropriate, and statistics/charts.

IMPORTANT: Frontend static/demo numbers must eventually be replaced with live backend/database data.

6. CURRENT BACKEND STRUCTURE

```
backend/
|
+-- config/
|   +-- db.js
|
+-- controllers/
|   +-- authController.js
|   +-- claimController.js
|   +-- predictionController.js
|
+-- routes/
|   +-- authRoutes.js
|   +-- claimRoutes.js
|   +-- predictionRoutes.js
|
+-- middleware/
|   +-- authMiddleware.js
|
+-- models/
|   (planned / may be expanded)
|
+-- services/
|   (planned / may be expanded)
|
+-- .env
+-- package.json
+-- package-lock.json
```

```
+- server.js
+- node_modules/
```

7. DATABASE — CURRENT WORKING STATE

MySQL Server is confirmed running. MySQL Workbench is confirmed connected. The database insurance_fraud_detection exists and has been selected successfully. The following six tables have been created and tested.

Database:
insurance_fraud_detection

Tables:
1. users
2. policies
3. claims
4. documents
5. ai_predictions
6. audit_logs

8. EXACT DATABASE SQL CURRENTLY IMPLEMENTED

```
CREATE DATABASE insurance_fraud_detection;
USE insurance_fraud_detection;
```

```
CREATE TABLE users (
  user_id INT AUTO_INCREMENT PRIMARY KEY,
  full_name VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  phone VARCHAR(20),
  password_hash VARCHAR(255) NOT NULL,
  role ENUM('user', 'admin', 'inspector') DEFAULT 'user',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
  ON UPDATE CURRENT_TIMESTAMP
);
```

```
CREATE TABLE policies (
  policy_id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  policy_number VARCHAR(50) NOT NULL UNIQUE,
  policy_type VARCHAR(100),
  start_date DATE,
  expiry_date DATE,
  premium_amount DECIMAL(12,2),
  coverage_amount DECIMAL(12,2),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

  FOREIGN KEY (user_id)
    REFERENCES users(user_id)
    ON DELETE CASCADE
);
```

```
CREATE TABLE claims (
  claim_id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  policy_id INT NOT NULL,
  claim_amount DECIMAL(12,2) NOT NULL,
  claim_type VARCHAR(100),
  hospital_name VARCHAR(200),
  vehicle_number VARCHAR(50),
  vehicle_age INT,
  police_report BOOLEAN DEFAULT FALSE,
  incident_date DATE,
  incident_location VARCHAR(255),
  description TEXT,
  claim_status ENUM(
    'pending',
    'approved',
    'rejected',
    'under_review'
  ) DEFAULT 'pending',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
  ON UPDATE CURRENT_TIMESTAMP,

  FOREIGN KEY (user_id)
```

```

        REFERENCES users(user_id),

    FOREIGN KEY (policy_id)
        REFERENCES policies(policy_id)
);

CREATE TABLE documents (
    document_id INT AUTO_INCREMENT PRIMARY KEY,
    claim_id INT NOT NULL,
    document_type ENUM(
        'medical_bill',
        'fir',
        'insurance_document',
        'image',
        'other'
    ),
    file_name VARCHAR(255),
    file_path VARCHAR(500),
    uploaded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (claim_id)
        REFERENCES claims(claim_id)
        ON DELETE CASCADE
);

CREATE TABLE ai_predictions (
    prediction_id INT AUTO_INCREMENT PRIMARY KEY,
    claim_id INT NOT NULL,
    prediction ENUM('fraud', 'genuine') NOT NULL,
    fraud_probability DECIMAL(5,4),
    genuine_probability DECIMAL(5,4),
    risk_level ENUM(
        'low',
        'medium',
        'high'
    ),
    model_version VARCHAR(50),
    prediction_reasons TEXT,
    predicted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (claim_id)
        REFERENCES claims(claim_id)
        ON DELETE CASCADE
);

CREATE TABLE audit_logs (
    log_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT,
    claim_id INT,
    action VARCHAR(100),
    description TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (user_id)
        REFERENCES users(user_id)
        ON DELETE SET NULL,

    FOREIGN KEY (claim_id)
        REFERENCES claims(claim_id)
        ON DELETE SET NULL
);

```

9. DATABASE TEST DATA CURRENTLY INSERTED

```
INSERT INTO users
(full_name, email, phone, password_hash, role)
VALUES
('Test User', 'test@example.com', '9876543210',
'test_password_hash', 'user');

INSERT INTO policies
(user_id, policy_number, policy_type, start_date, expiry_date,
premium_amount, coverage_amount)
VALUES
(1, 'POL-10001', 'Health Insurance', '2026-01-01',
'2026-12-31', 15000, 500000);

INSERT INTO claims
(user_id, policy_id, claim_amount, claim_type, hospital_name,
police_report, incident_date, incident_location, description)
VALUES
(1, 1, 125000, 'Medical', 'ABC Hospital', TRUE,
'2026-07-10', 'Pune', 'Test insurance claim');

Verification:
SELECT * FROM users;
SELECT * FROM policies;
SELECT * FROM claims;

Schema verification:
DESCRIBE users;
DESCRIBE policies;
DESCRIBE claims;
DESCRIBE documents;
DESCRIBE ai_predictions;
DESCRIBE audit_logs;
```

These database operations were reported as working successfully. Note: the inserted test password is only a placeholder test value. Real registration must use bcrypt hashing.

10. WORKING NODE.JS DATABASE CONFIGURATION

Current config/db.js:

```
const mysql = require("mysql2/promise");
require("dotenv").config();

const pool = mysql.createPool({
  host: process.env.DB_HOST,
  port: process.env.DB_PORT,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0
});

module.exports = pool;
```

11. ENVIRONMENT CONFIGURATION

```
PORT=5001

DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD=YOUR_ACTUAL_MYSQL_PASSWORD
DB_NAME=insurance_fraud_detection

JWT_SECRET=your_super_secret_key_change_this
```

The real MySQL password is stored locally by the user and must never be requested in chat or placed into shared documentation. The .env file must remain private and should be added to .gitignore.

12. WORKING SERVER.JS FOR CURRENT BACKEND/DB TEST

```
const express = require("express");
const cors = require("cors");
require("dotenv").config();

const db = require("../config/db");

const app = express();

app.use(cors());
app.use(express.json());

app.get("/", (req, res) => {
  res.status(200).send(
    "Healthcare Fraud Detection Backend is Working!"
  );
});

app.get("/api/health", (req, res) => {
  res.status(200).json({
    status: "OK",
    message: "Backend is running"
  });
});

app.get("/api/health/db", async (req, res) => {
  try {
    const [rows] = await db.query(
      "SELECT 1 AS database_connection"
    );

    res.status(200).json({
      status: "OK",
      message: "MySQL database connected successfully",
      database: process.env.DB_NAME,
      result: rows
    });

  } catch (error) {
    console.error(
      "Database connection error:",
      error.message
    );

    res.status(500).json({
      status: "ERROR",
      message: "MySQL database connection failed",
      error: error.message
    });
  }
});

const PORT = process.env.PORT || 5001;

app.listen(PORT, () => {
  console.log("=====");
  console.log("Backend started successfully");
  console.log(`http://localhost:${PORT}`);
  console.log("=====");
});
```

13. CONFIRMED WORKING TESTS

Start:
node server.js

Expected terminal:
=====

```
Backend started successfully
http://localhost:5001
=====
```

GET http://localhost:5001/
Expected:
Healthcare Fraud Detection Backend is Working!

GET http://localhost:5001/api/health

Expected:

```
{
  "status": "OK",
  "message": "Backend is running"
}
```

GET http://localhost:5001/api/health/db

Expected:

```
{
  "status": "OK",
  "message": "MySQL database connected successfully",
  "database": "insurance_fraud_detection",
  "result": [
    {
      "database_connection": 1
    }
  ]
}
```

The Node.js → mysql2 → MySQL → insurance_fraud_detection connection has been confirmed working.

14. CURRENT STATUS — DO NOT CONFUSE COMPLETED AND PENDING

Component	Status
Project concept and architecture	COMPLETED / DEFINED
React frontend UI	MOSTLY COMPLETED / UI CREATED
AI Fraud Detection UI	DESIGNED / INCLUDED
MySQL Server + Workbench	COMPLETED
insurance_fraud_detection database	COMPLETED
Six database tables	COMPLETED
Database relationships	IMPLEMENTED
Test user/policy/claim data	WORKING
Backend folder structure	CREATED
config/db.js	WORKING
Environment configuration	CONFIGURED
Express server	WORKING on port 5001
Backend health endpoint	WORKING
Node.js → MySQL connection	COMPLETED + VERIFIED
Authentication + JWT	PENDING / NEXT
Claim CRUD APIs	PENDING
File upload	PENDING
AI/ML fraud model	PENDING
Python AI API	PENDING
Node → Python AI integration	PENDING
React → Backend integration	PENDING
Dynamic dashboard	PENDING
Admin backend workflow	PENDING
Audit logging integration	PENDING
Blockchain	PLANNED LATER
End-to-end testing	PENDING
Deployment	NOT STARTED

15. NEXT FEATURE — AUTHENTICATION

The next development step is authentication. The project already contains `authController.js`, `authRoutes.js`, and `authMiddleware.js`. The new chat must inspect those files first and should not overwrite them without reviewing their current content.

Target authentication flow:

```
REGISTER
User submits:
full_name
email
phone
password
    |
    v
Validate input
    |
```

```

      v
Check duplicate email
      |
      v
Hash password with bcrypt
      |
      v
INSERT into users
      |
      v
Return success response

LOGIN
User submits:
email
password
      |
      v
Find user by email
      |
      v
Compare password using bcrypt
      |
      v
Generate JWT
      |
      v
Return token + user info

PROTECTED PROFILE
Client sends:
Authorization: Bearer <JWT>
      |
      v
authMiddleware verifies token
      |
      v
GET /api/auth/profile
      |
      v
Return authenticated user profile

```

Target endpoints:

```

POST /api/auth/register
POST /api/auth/login
GET  /api/auth/profile

```

Authentication test checklist:

1. Register valid user
2. Confirm user saved in MySQL
3. Confirm password is bcrypt hash, not plaintext
4. Try duplicate email
5. Try invalid registration data
6. Login with correct credentials
7. Confirm JWT is returned
8. Login with wrong password
9. Access profile with valid JWT
10. Access profile without JWT
11. Access profile with invalid JWT
12. Confirm protected route behavior

16. CLAIM FEATURE AFTER AUTHENTICATION

Target endpoints:

```

POST  /api/claims
GET   /api/claims
GET   /api/claims/:id
PUT   /api/claims/:id
DELETE /api/claims/:id

```

Claim workflow:

```

Authenticated User
|
v

```

```

Submit Claim
|
v
Validate user + policy
|
v
Save claim in claims table
|
v
Return claim_id
|
v
Upload supporting documents
|
v
Store document metadata
|
v
Trigger AI prediction
|
v
Save prediction
|
v
Show result to user

```

17. DOCUMENT UPLOAD FEATURE

Use multer for file uploads. The documents table stores metadata, while actual files may be stored locally or in cloud/object storage. Do not store sensitive healthcare documents directly in the database unless there is a specific reason.

Supported document types in current schema:

- medical_bill
- fir
- insurance_document
- image
- other

Stored metadata:

```

document_id
claim_id
document_type
file_name
file_path
uploaded_at

```

18. AI/ML FRAUD DETECTION FEATURE

The AI component is planned to analyze claim-related features and predict whether a claim is potentially fraudulent or genuine. It should provide probabilities, risk level, and supporting reasons/indicators where possible.

```

Dataset
|
Data Cleaning
|
Exploratory Data Analysis
|
Feature Engineering
|
Train/Test Split
|
Model Training
|
Model Evaluation
|
Save Model
|
Prediction API

```

Candidate models: Logistic Regression, Decision Tree, Random Forest, and XGBoost. The final model must be selected based on evaluation results rather than assuming one model is best.

19. PYTHON AI SERVICE

Planned structure:

```
ai_service/  
|  
+-- model.py  
+-- train.py  
+-- predict.py  
+-- app.py           # Python API entry point  
+-- requirements.txt  
+-- model_artifact   # saved trained model
```

Flow:

```
Node.js Backend  
  |  
  | claim features  
  v  
Python AI API  
  |  
  v  
Saved ML Model  
  |  
  v  
Fraud probability  
Genuine probability  
Risk level  
Prediction reasons  
  |  
  v  
Node.js Backend  
  |  
  v  
ai_predictions table
```

20. NODE → AI INTEGRATION

```
POST /api/predictions  
  |  
  v  
Node.js receives claim_id  
  |  
  v  
Load claim features  
  |  
  v  
Call Python AI API  
  |  
  v  
Receive prediction  
  |  
  v  
Save to ai_predictions  
  |  
  v  
Return prediction to React
```

21. REACT → BACKEND INTEGRATION

```
React  
  |  
  | Axios  
  v  
Node.js REST API  
  |  
  v  
MySQL / Python AI  
  |  
  v  
JSON response  
  |  
  v  
React state / Context  
  |  
  v  
Dashboard / Claims / Prediction UI
```

Static frontend dashboard values must eventually be replaced with real API responses. Authentication should use AuthContext and JWT token handling. ProtectedRoute should prevent unauthorized access to protected pages.

22. ADMIN WORKFLOW

```
Admin Login
|
v
Admin Dashboard
|
+--> Search claims
+--> Filter claims
+--> View claim details
+--> View documents
+--> View AI prediction
+--> Approve claim
+--> Reject claim
+--> Send for review
+--> View users
+--> View statistics
|
v
Record action in audit_logs
```

23. AUDIT LOGGING

Important actions should be recorded, including claim submission, administrative decisions, viewing activity where appropriate, and AI prediction generation. The audit_logs table links actions to users and claims.

24. BLOCKCHAIN — LATER ONLY

Blockchain is not part of the current core implementation. It should only be added after React → Node.js → MySQL → AI is stable and tested. A practical approach is to generate a hash of selected claim/prediction data and record the hash and timestamp on a blockchain for integrity verification. Sensitive healthcare documents should not be placed directly on-chain.

25. FULL DEVELOPMENT ORDER

PHASE 1	Database implementation	[DONE]
PHASE 2	Node.js + MySQL integration	[DONE - STEP 1]
PHASE 3	Authentication + JWT	[NEXT]
PHASE 4	Claim CRUD APIs	[PENDING]
PHASE 5	Document uploads	[PENDING]
PHASE 6	AI/ML model development	[PENDING]
PHASE 7	Python AI prediction API	[PENDING]
PHASE 8	Node.js -> Python AI integration	[PENDING]
PHASE 9	React -> Backend API integration	[PENDING]
PHASE 10	Dynamic dashboard	[PENDING]
PHASE 11	Admin workflow + audit logs	[PENDING]
PHASE 12	Full end-to-end testing	[PENDING]
PHASE 13	Optional blockchain integrity	[PLANNED]
PHASE 14	Deployment	[PENDING]

26. DEFINITION OF FINAL COMPLETION

The project is functionally complete when a user can register and log in, submit an insurance claim with supporting documents, store the claim in MySQL, have the claim analyzed by an ML model, receive fraud/genuine prediction with risk information, view the result in React, and allow an authorized administrator to review and update the claim while maintaining an audit trail.

27. FIRST MESSAGE TO SEND IN THE NEW CHAT

I have uploaded the MASTER PROJECT CONTEXT file for my Healthcare Insurance Fraud Detection System.

Read the entire file before starting.

This is an existing project. Do not rebuild it from scratch.

I want you to continue development from the exact current state described in the file.

First:

1. Confirm your understanding of the architecture.
2. Confirm what is completed.
3. Confirm what is pending.
4. Do not overwrite existing files blindly.
5. Ask me to provide the current contents of `authController.js`, `authRoutes.js`, and `authMiddleware.js`.
6. Review those files.
7. Then implement the Authentication feature step-by-step:
Register -> bcrypt password hashing -> MySQL -> Login -> JWT -> protected Profile.
8. Give me exact code and exact commands.
9. Test each step before continuing.

Do not move to the next major feature until I confirm the current feature works.

28. IMPORTANT SECURITY / DEVELOPMENT RULES

- Never expose the real MySQL password or JWT secret in chat, documentation, screenshots, GitHub, or public repositories.
- Add `.env` to `.gitignore`.
- Never store plaintext passwords. Use bcrypt.
- Use JWT only after successful credential verification.
- Validate and sanitize API input.
- Use parameterized SQL queries to reduce SQL injection risk.
- Protect private routes with authentication middleware.
- Add role-based authorization for admin and inspector actions.
- Validate uploaded files and restrict file size/types appropriately.
- Do not put sensitive healthcare documents directly on blockchain.
- Do not mark a feature complete until it has been tested.
- Maintain backward compatibility with already working features whenever practical.

END OF MASTER CONTEXT

This file is intended to be uploaded to a new ChatGPT conversation as the primary project context for continuing implementation.